Exercise 7 — Decision Trees and Ensembles

Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Set 6 November 2026 · discussed at the start of lesson 8, Friday 13 November 2026

Contents

What this exercise is for

Lesson 7's own dataset was clear-cut: every ensemble — bagging, random forest, gradient boosting — beat the best single tree, consistently, by about three points of cross-validated accuracy or more. It would be easy to leave the lecture believing an ensemble always beats a single tree by *some* margin.

That is not true, and this exercise is built to show you when and why.

You get **two loan products from the same lender**, built the same way a rule, applied, then a slice of labels flipped to stand in for outcomes the on-file features cannot explain. On one product, a single depth-tuned tree comes within noise of the best ensemble. On the other, the ensemble wins by a wide and reproducible margin. Neither ordering can be guessed from the names of the algorithms; both can be predicted from twenty minutes spent looking at the data, which is what makes it a decision tree lesson and not a coin flip.

That is the skill being assessed. Not fitting models — **reading a rule well enough to know, before you fit anything, whether it needs an ensemble.**

Keep a single notebook, `exercise07.ipynb`, that runs top to bottom in the course container. Written answers go in markdown cells beside the evidence for them.

Nothing is handed in: lesson 8 opens by discussing this exercise, and at the exam one of the ten is drawn for you to talk through your own notebook.

The two products

`loan_products_data.py` ships with this exercise and needs no network.

```
from loan_products_data import load_personal_loan, load_equipment_loan

X_p, y_p = load_personal_loan()      # retail lending, fixed schedule
X_e, y_e = load_equipment_loan()     # commercial lending against machinery
```

Personal loan. Two figures on file: applicant income and how much of their existing revolving credit they have already drawn down. Underwriting is a checklist that reads one factor at a time: an income floor, and a utilisation ceiling, applied independently.

Equipment loan. Three figures on file: the borrower's annual revenue, its debt-service coverage ratio (DSCR — free cash flow divided by the debt payment it must cover), and a sector-cyclicality score. There is no single floor or ceiling on any one of the three. Underwriting turns on which *combination* of size, coverage and sector a business falls into.

Read the module docstring before you do anything else. It describes both products' underwriting logic honestly and at length, and it is not padding — everything you need to *predict* Part 1's answer is in it.

`loan_products_data.py` also carries `TRUE_*` constants recording exactly how each rule was built, feature by feature. **Do not read them until Part 4.**

Both loaders take `n_samples` and `random_state`; leave both at their defaults unless a part below asks you to vary them.

Part 1 — Predict, before you fit anything (20 marks)

1. Load both products and report, for each: the number of rows, the number of columns, the positive rate, and the majority-class baseline.
2. Look at the data. Plot whatever helps — a scatter of the two personal-loan features coloured by outcome is one obvious start; the equipment loan has three features, so you will need more than one view (pairwise scatter plots, or a plot per feature against the outcome). Do **not** fit a model yet.
3. Then, in writing, **predict which will win on each product**: a single depth-tuned decision tree, or an ensemble (a random forest, gradient boosting, or both). Give a reason per product, in terms of lesson 7:
   - How many separate regions does the risky class seem to occupy, as best you can tell from the plots?
   - Does isolating the risky region look like it needs one feature at a time, or more than one feature agreeing simultaneously?
   - How much data looks like it sits behind each boundary you can see?

What is being marked: the reasoning, not the ranking. A confident wrong prediction that engages with the three questions above scores well; a hedged answer that commits to nothing does not. **Write the prediction down before you run anything** — the rest of the exercise is much less instructive if you do not.

Part 2 — Measure (25 marks)

4. At each product, cross-validate at least: the majority-class baseline, a single decision tree with its depth chosen by cross-validation (report which depth won and how you chose it), and at least one ensemble method (a random forest, gradient boosting, or both). Report accuracy with its spread across folds — the machinery of lesson 5, not a single split.
5. Present the two products side by side, baseline included.
6. State plainly where your Part 1 predictions were right and where they were wrong.

What is being marked: that the comparison is honest — the tree's depth is chosen by cross-validation rather than guessed, the same folds are used for every model at a given product, and question 6 is answered rather than quietly skipped.

One warning, worth more than it looks: if you fit a `GradientBoostingClassifier` with its defaults and it does not look like it is winning anywhere, check its `max_depth` before you conclude anything. Lesson 7 section 8 built gradient boosting out of *shallow* trees on purpose — but a weak learner that is too shallow to represent a rule needing several features to agree at once will need more rounds, or deeper individual trees, to catch up. Working that out is worth a sentence in Part 3, not a shrug.

Part 3 — Explain the inversion (30 marks)

The ranking is not the same at the two products. Explain why, in a paragraph of six to ten sentences per product.

Support each explanation with a **measurement**, not an assertion. Suggestions, though you may find better ones:

- for a claim that a boundary needs several features to agree at once: fit a tree (or the ensemble) on one feature at a time, then on all of them, and compare accuracy;
- for a claim about how many disjoint regions the risky class occupies: look at how many leaves, or what depth, a single tree needs before its training accuracy stops climbing sharply — section 3's depth-versus-accuracy table is the pattern;
- for a claim about instability: refit the single tree on a handful of independent resamples of the same product (as in section 4) and measure how much its predictions, or its chosen splits, actually move;
- for a claim that averaging is or is not doing useful work: compare a random forest's out-of-bag score against its cross-validated accuracy — section 5.1's agreement to four decimal places is what "doing nothing extra" looks like, and a large gap between the two is worth noticing.

What is being marked: this part carries the most marks in the exercise, and the measurements are what separate an explanation from a plausible story. Lesson 7's handout makes several claims of exactly this kind and backs every one with a number computed a second way; do the same.

Part 4 — Mark yourself (25 marks)

Now open `loan_products_data.py` and read the `TRUE_*` constants.

7. At the personal loan: how many conditions does the true rule actually need, and on how many features? Compare that against the tree depth cross-validation chose in Part 2 — is the tree spending more splits than the rule needs, fewer, or about the right number?
8. At the equipment loan: how many stress pockets does `TRUE_STRESS_POCKETS` define, and how many features must agree at once for each one? Does that match what your Part 3 measurements suggested, or did you underestimate or overestimate the rule's complexity?
9. Both products flip a fraction of labels after the rule is applied (`TRUE_LABEL_NOISE_PERSONAL`, `TRUE_LABEL_NOISE_EQUIPMENT`). What is the resulting noise ceiling at each product, and how close did your best model get to it?
10. Having seen the true rule at each product, explain in two or three sentences *why* an ensemble does or does not help represent it — referring to lesson 7's variance-floor argument (section 5.2) or its depth-as-bias-variance-dial argument (section 3), whichever actually applies.

What is being marked: question 10. Being able to look at a generating rule and say *why* a tree, or an average of many trees, could or could not represent it economically is the whole content of this lesson.

Marking

Part	Marks
1 — Predict before fitting	20
2 — Measure	25
3 — Explain the inversion	30
4 — Mark yourself	25
Total	**100**

Marks are lost for:

- reporting a single-split score instead of a cross-validated one (**—10**);
- choosing a tree depth by eye rather than by cross-validation, or not reporting which depth was chosen (**—10**);
- declaring a winner at the personal loan without acknowledging the width of the error bars there (**—5**);
- asserting a claim in Part 3 with no supporting number from your own run (**—5** per unsupported claim);
- a notebook that does not run top to bottom in the container (**—10**).

Marks are **not** lost for a wrong prediction in Part 1, nor for a conclusion of "the tree and the ensemble are not distinguishable at this sample size," provided that is honestly what your numbers show — lesson 5's methodology still applies here. A wrong prediction you then investigated is worth more than a cautious one that hedged.

Getting help

Email fabio.antonini.1969@gmail.com. There is no lesson between the day this is set and the morning it is due, so the inbox is the only channel. If you are stuck on Part 3, notebook 2 of lesson

7 does exactly this kind of measurement for the lesson's own dataset — it claims a random forest decorrelates its trees, and then measures the tightening spread across resampled splits to show it.